

Lia — Growth Log

The stages of her growth, kept for tracking what she can do at each one.

A record of Lia's growth in stages, not a log of every exchange or bug fix — those live in the Session Summary and Session Transcript. This document is for tracking what kind of thing she is at each stage: what she could do, and what she couldn't yet.

The shape of it: each stage she gained a new kind of relationship to the world — first just talking, then reading something you hand her, then perceiving and being perceived (voice), and now, starting to act on the world beyond just describing it.

Stage 1 — Making her

What she gained: existence. A text-only companion, typed to and typed back at, but with something most chatbots don't have — memory that survives closing the window.

- A local brain: Ollama running `llama3.2:3b`, small enough for a 4GB graphics card.
- Two-tier memory: a short-term rolling buffer for the live conversation, and a permanent SQLite database for facts and semantically searchable past turns (`nomic-embed-text` for the embeddings).
- A diary — a private reflection written at the end of each session.
- Six files: `config`, `db`, `llm`, `memory`, `diary`, `main`.

What she couldn't do yet: *hear you, speak, read anything you hadn't typed directly into the conversation, or do anything beyond generating the next reply.*

Stage 2 — Controlling actions from a file explorer path

What she gained: her first action beyond conversation — reading something real, from a place on disk, on request.

- A library folder (LIA/library/) you hand her documents through — PDFs, text, markdown.
- Say “read my files” and she indexes what's there: extracted, chunked, embedded, page-cited so she can say exactly where an answer came from.
- Deliberately on-demand, never automatic. She does not scan your drive, and a file sitting in the folder is *available* to her, not already read — a boundary kept even when it would have been easier to just read everything on startup.

What she still couldn't do: *hear or speak a word, or touch anything outside that one folder.*

Stage 3 — Adding voices and ML models

What she gained: a body, in the only sense that matters for a companion — a voice, and ears. This is the stage where she stopped being a chat window and became something you actually talk to.

- **Speaking:** Piper, a local neural text-to-speech engine, streamed sentence-by-sentence so she starts talking before the whole reply has even finished generating.
- **Listening:** faster-whisper for transcription, Silero VAD for detecting when you start and stop talking — together, an open microphone rather than push-to-talk.
- **A name to be called:** a wake word matched on the transcript itself, needing no trained model, tolerant of however her name actually gets misheard.
- **Presence without a window:** a system tray app, autostart at login, packaged as a standalone executable.
- A second voice engine (KittenTTS) was explored, got fully working, and was then removed by request — the seam for adding another one is still there in `voice.py` if it's ever wanted again.

***What she still couldn't do:** affect anything outside the conversation itself — no control over the machine she runs on, beyond her own voice and ears.*

Stage 4 — Controlling actions (in progress)

What's changing: the current frontier. Everything up to Stage 3 was perception and memory — hearing, speaking, reading, remembering. Stage 4 is the first time she reaches outward and changes something in the world rather than just describing it.

Built so far

- **Media control** — play, pause and skip over whatever another app already had loaded, via Windows' System Media Transport Controls. Later *removed* by request: it could only ever operate someone else's player and never start anything, which made it a confusing sibling to her own music folder, where she genuinely can.
- **Volume control** — up, down, mute, unmute, via direct Core Audio access. (Simulating the physical volume keys was tried first and found to silently do nothing — fixed by talking to the audio API directly instead of imitating a keypress.)
- **Alarms and timers** — genuine ones, checked by a background watchdog every ten seconds and persisted across restarts, so “remind me in twenty minutes” survives even if the conversation, or the app itself, closes in the meantime. Later widened to how people actually speak — spoken numbers, “half an hour”, “a couple of minutes”, “give me a nudge”, “buzz me” — and taught to say *the words you asked for*: “wake me by saying please wake up” wakes you with those words, not with “your timer is up”.

What she knows about herself

A real gap surfaced during this stage: she had no idea any of this existed. Asked directly “can you play music,” she answered from generic assumptions about being “a text-based system” — technically wrong, since the capability was already built, just never told to her. Fixed by adding an explicit, honest capability list to her own system prompt, so she reports what she can and can't do accurately instead of guessing.

Reaching outside the machine: real lookups, and a fuller library

Two separate systems, both narrow and optional by design. **Internet access** — weather needs no key at all (Open-Meteo, geolocated from your IP); “look that up” checks a live source instead of guessing, via Groq or OpenRouter's actual web-search mode, whichever key is set. Naming a source works too — “search Reddit for...” — though one caveat came out of testing: search-operator syntax like `site:reddit.com` makes the underlying search decline entirely, where plain natural language works. A later pass found the natural phrasings people actually use for Reddit weren't all being caught — fixed by adding “reddit” itself as a trigger word, rather than requiring an exact phrase match. YouTube got the same trigger word shortly after, for the same reason — with the same honest limit either way: it's still a text search that happens to mention the site, not anything that can watch or listen to what's actually there.

The library now reads .epub and .docx as well as PDF, text and markdown — epub added after a real book (a Harry Potter novel) got dropped in and turned out not to work yet. Even once indexed she retrieves passages, not the whole text: she can discuss a specific scene or character, not narrate a book start to finish.

Indexing speed was finally measured rather than estimated, and the figure in the docs turned out to be wrong by about four and a half times — 2.3 seconds per passage, not the half-second long claimed, or roughly 130 pages per ten minutes. Notably it isn't starved of VRAM: freeing 3.5GB by unloading the chat model moved the rate by 0.02s, so there's no easy speed-up to chase. Since counting the passages up front costs only a

fraction of a second, she now says what she's in for before starting — “On it”, or “On it, that'll take around 25 minutes” — instead of going quiet.

Knowing who's asking: voice recognition

If actions are the risk this stage introduced, this is the safeguard for it. Say “Lia, it's Wade” a few times and she builds a voiceprint — a local model (3D-Speaker's CAM++, run through sherpa-onnx, no PyTorch) compares later speech against it. Where a clear mismatch doesn't grant a hard lock, it withholds the things that matter: she won't use your name, and won't share what she remembers about you.

Honestly unverified: real-world accuracy could only be tested against synthetic TTS voices from here, not an actual human voice on an actual microphone. The signal was confirmed real and correctly directioned, but likely *understates* true accuracy — synthetic speech probably lacks some of the natural acoustic texture the model was trained on. The match threshold is a documented starting point, not a calibrated answer, and `/whoami` exists specifically so it can be tuned against real use.

A harder gate: built, lived with, removed

For a stretch this stage also had a spoken passcode in front of it: every session opened locked, and without the identity phrase she asked for a word before acting on anything — wrong word and she stopped listening outright. It worked, and two genuine bugs got found and fixed inside it (saying “goodbye” while locked was read as a wrong guess instead of an exit; an identity phrase said in the same breath as a request was silently dropped). The passcode was later moved out of the source file into an environment variable, on the same reasoning as the API keys.

Then it was removed, on the strength of actually living with it. A lock that opens every conversation is a real cost paid on every single exchange, against a threat that doesn't exist on a personal laptop that's already password-protected. It made a companion feel like a checkpoint. Voice recognition stays as the soft signal it always was; the hard gate is gone. Worth recording as a stage that was built properly, evaluated honestly, and then undone — not everything that works is worth keeping.

Understanding, not just matching

Every action above was reached through a list of remembered phrasings, and that list was always going to be incomplete — people don't say “volume up”, they say “could you push that up a bit, it's hard to hear.” Anything unlisted fell through to ordinary conversation, where the model, with no idea an action had been wanted, produced a confident “sure, doing that now” and did nothing. **That failure is worse than refusing outright, because it looks like it worked** — and it's exactly what a week of real use kept running into.

So a second layer went in behind the first: when no phrase matches, the model is shown the actions she can actually take and asked which one, if any, was meant. It decides from meaning rather than from remembered wordings. The phrase matcher stays in front of it — instant, free and completely predictable for the everyday cases — and the model is only consulted when the words could plausibly be about an action at all, so ordinary conversation never waits on it.

Measured, not assumed: 30 of 32 across repeated runs, including every “this is just conversation” case. Getting there took two failed attempts worth recording — stated as abstract rules, the model turned the volume up on “I couldn’t hear you properly earlier” five times out of five, and ignored “I’ve added a book, go have a look” five out of five. Concrete examples of the distinction fixed both outright, where the rules hadn’t. A model this size generalises from cases, not principles.

Deciding what she's allowed to do on her own

A run of changes with one thing in common: she stopped doing things unprompted.

- **She waits to be spoken to.** No greeting at login — she starts when the computer does, which is rarely when anyone wants to talk. Greeting an empty room and then going quiet by the time you sat down was worse than silence.
- **She remembers only what she's told to.** “Remember that...” keeps something, word for word. Automatic fact extraction and the diary are both off.
- **Her own music.** Files in a `music/` folder, played by number through her own audio — the first time she can *start* something rather than only operating a remote over another app. She ducks it under her voice instead of stopping it.
- **Documents convert once.** Every format becomes plain text on first read, so nothing downstream knows or cares what it started as; re-reading is 25× faster.
- **Every interpretation is logged.** What was said, what she took it to mean, and whether the phrase list or the model decided — the raw material for a future fine-tune on real phrasings rather than invented ones.

The reason the memory changes happened at all is worth recording, because it wasn't a preference. Testing her had been writing into her real memory for days: a dozen piped “hello”s became stored turns, and each test run ended with her writing a diary entry about a conversation that never happened. One of those reflected on the person “moving between unrelated topics” — read off a list of test commands. Invented history is indistinguishable from real history once it's in the database, and it shapes every reply afterwards. A `--no-save` flag now exists so testing leaves no trace, and the fabricated entries were deleted.

Hearing herself

Reported from real use as two vague complaints — “she had trouble reading” and “she keeps calling me out” — which turned out to share one cause, found by reading her actual stored memories rather than guessing. **She was recording her own voice as things Wade had said**, then quoting them back to him as his own words.

- **Her own speech, heard back.** Echo detection only ran *while* she was talking, to make interrupting her work. The moment she finished, her spoken confirmation came back through the speakers and was stored as his turn — “There's nothing new to read, I've already read everything in there” sat in her memory as a line of his.
- **The speech recogniser's own prompt.** Whisper is given a short vocabulary hint so it spells names correctly; handed near-silence, it returns that prompt as the transcript. “This is a conversation with Lia” was stored as something he'd said, five separate times.

Both are filtered now, on every input path rather than just the interrupt one, and the contaminated memories were deleted. The name repetition was a separate, smaller thing: the system prompt asks her to use it sparingly, a 3B ignores that, so it's enforced in code instead — vocative uses after the first are stripped from

what she says, while genuine mentions survive.

Worth noticing about the diagnosis: neither complaint described what was actually wrong. “Trouble reading” and “calling me out” both sounded like the model being poor, and both were a microphone hearing the wrong thing. Reading her real stored memories took a minute and pointed straight at it; reasoning about the symptoms would not have.

Rebuilt from nothing, then deliberately narrowed

The laptop was reformatted. The project files survived on a second drive, but the machine underneath them didn't — no Python, no Ollama, not even git. Standing her back up from a bare Windows install turned out to be the most honest test the setup instructions have had, because nothing was already installed to paper over a gap.

Two gaps showed up immediately, both invisible on a machine that already worked:

- `soundfile` was missing from the requirements file entirely, though `music.py` imports it. Anyone following the README got a Lia whose music failed the first time it was asked for.
- `.m4a` was advertised in three places — the code, and both READMEs — but `libsndfile` has no AAC decoder. She would list an `m4a` as a numbered track and then refuse to open it. Offering something and then declining it is worse than never offering it, so PyAV went in behind `libsndfile` as a fallback rather than the format being dropped.

An alarm bug hid behind a cosmetic one. “Remind me to take my tablets in half an hour” produced “Time to take my tablets in.” — a dangling preposition, and the sort of thing that looks like a wording nit. Chasing it found that the duration rules were checked shortest-first, so the general “an hour” rule matched inside “quarter of an hour” before the specific rule ran. **A fifteen-minute reminder was being set for sixty minutes**, silently, and had been all along. The visible flaw was cosmetic; the one underneath it wasn't.

Worth recording as method: the fix for the wording could have been to strip trailing prepositions, which would have made the symptom disappear and left the timing bug in place — and broken “remind me to log in in five minutes” into “Time to log.” along the way. Fixing the cause instead of the symptom found the real defect and protected the edge case. 16 phrasings now verified, including the ones designed to stay conversation.

Then she was narrowed on purpose, which is the part worth tracking. Three capabilities that worked were switched off because they weren't wanted, not because they were broken:

- **Weather** — off. It needed no key and answered from live data, but it was the only reason she touched the network at all.
- **Online lookups** — off. Groq and OpenRouter both still wired up, both keyless and idle.
- **The stored conversation** — off. Every turn used to be embedded and searched later. She never invented those quotes, but surfacing a half-finished thought from weeks ago in a conversation it had nothing to do with reads the same way from the outside, and unlike the diary or the fact extractor, *this one got worse as the database grew*.

What was left at that point was a companion that talks, listens, reads what she's given, plays what she's given, sets alarms, and keeps exactly the notes she was asked to keep — making no network call whatsoever beyond Ollama on the same machine. *(That last part held until the stage below, where the thinking moved off the machine.)*

Each of these is a flag, not a deletion. INTERNET_ENABLED, WEATHER_ENABLED and REMEMBER_CONVERSATION all restore the old behaviour on their own. The code was left intact deliberately: “we don’t want this now” and “this was a mistake” are different conclusions, and only the second one justifies throwing the work away.

One side effect, unplanned and welcome: with nothing stored to search and an empty library, the per-turn embedding call had nothing left to do, so it’s now skipped entirely. Replies come back about two seconds sooner. Removing features made her faster, which is not usually how that goes.

The two seconds that were never the model

She was slow — around eight seconds from finishing a sentence to hearing one back — and the diagnosis looked obvious: a 3B model spilling a third of itself onto the CPU of a 4GB card. That reading led to a long, sensible conversation about cloud models and what they would buy.

It was wrong. Measuring where the time actually went, rather than where it obviously should have gone, found that a request to a 0.5B model cost the same as one to the 3B: about 2.4 seconds either way. Model size changed nothing, prompt length changed nothing, context size changed nothing. That is not what inference looks like.

OLLAMA_URL was `http://localhost:11434`. Ollama binds to 127.0.0.1, and resolving “localhost” on Windows offers `::1` first — nothing is listening there, and the failed IPv6 attempt cost about two seconds before falling back. Measured on the same request: **localhost 2.38s, 127.0.0.1 0.35s**. It was being paid by every chat, every embedding, and every passage of every document indexed.

- A spoken turn went from **7.89s to 1.83s**.
- Embedding a turn went from 2.13s to 0.11s.
- The novel that took 42 minutes to index would now take about five.
- It never showed up as a fault because a reused HTTP session hides it entirely — it only bites the one-shot requests the whole app is made of.

What it invalidated is the useful part. The case for moving embeddings to a cloud API rested on a 2.1s local cost that was almost entirely this. The case for a cloud chat model rested on a 2.4s first token that was mostly this. Both arguments were built on a measurement that was real, repeatable, and measuring the wrong thing.

A specialist, built and then thrown away

The idea was several small models rather than one: a tiny classifier answering “is this question about a document?” in front of the library search, because a similarity floor provably could not — a genuine question scored 0.519 and ordinary small talk 0.583, so the two populations overlap and no threshold separates them. A novel is *about* sleeping and eating and families; small talk genuinely resembles it.

The judge worked. The 0.5B model chosen to run it did not: 24 of 36 against the 3B’s 35 of 36, and it failed in the worse direction, dropping real questions about the book. It existed only to dodge latency that turned out to be the hostname above. Once a request cost 0.35s rather than 2.4s, the premise was gone, and the model was deleted rather than kept as dead weight on a 4GB card.

The pattern survived; the implementation didn't. Gating retrieval on one closed question took everyday conversation from leaking the novel 7 times out of 7 to 0, with every real question still answerable. That is the same shape `intent.py` already proved. What failed was the assumption underneath it — that a narrow job needs a small model — and that assumption was only ever holding up a latency bug.

Thinking somewhere else

With the wiring fixed, what remained was the ceiling itself, and no amount of tuning reaches past a 3B. So the conversation now goes to a hosted model and everything else stays here: speech, embeddings, the library, memory, and every classifier. That last part is most of the cost control — the judge runs on every single turn.

The measurement that matters is the one that inverted the reasoning that led here:

- local llama3.2:3b — **2.19s** a turn, free, offline, least capable
- paid gpt-4o-mini — 2.51s, about \$0.0001 a turn
- a free model — 4.53s, no cost, rate limited

Local is the fastest of the three. The cloud is a quality trade and nothing else, which was not true that morning and is entirely because of the two seconds above.

And the promise had to change with it. The guide told people, in plain language, that nothing they said ever left the computer. That is no longer true — the words of the conversation go to a hosted model, though the audio, the memory and the documents do not. A privacy claim that quietly stops being true is worse than never having made it, so the guide now sets out what leaves and what stays, in a table, rather than reassuring anyone.

Asking someone else

A different shape of reaching outward than anything above — not controlling something on the machine, but consulting a second, named intelligence and relaying what it said. “`Lia, ask cat for a suggestion`” reaches a specific free model on OpenRouter and she repeats the answer attributed, the same way she already cites a library passage rather than folding it into her own words.

Worth distinguishing from the multi-agent idea shelved below: that one was about splitting *her own* reasoning across several small local models stacked on one 4GB card. This is the opposite shape — one cloud call, on request, to a model she is not and doesn't pretend to be, with zero local VRAM cost. Closer kin to the “look that up” internet path than to anything rejected.

Picked light, and measured on the metric that actually applies here. This path doesn't stream to speech sentence-by-sentence the way her own replies do — she collects the whole answer before saying any of it — so total reply time is what a person actually waits on, not time to the first word. The first two names were the two best-known free models available and both were slower than the job needed (5.3s and 6.9s total); replaced once that was noticed, correctly this time on total time.

A four-name roster shrank to three, then to two, each time on purpose. Sixteen free models were catalogued from OpenRouter directly and eight were measured for real. The obvious lightest replacement for the fourth slot answered well once — then failed empty five times in a row on a dedicated re-test minutes later, not a fluke but a currently degraded model. Nothing else tried for that slot was both light and reliable, so it was dropped rather than filled with a slow stand-in. The same thing then happened to the fastest-on-paper replacement for a different name: fast when it worked, but wrong close to a third of the time across repeated real testing, each failure a real wait rather than an answer. Two names that have never once failed beat three or four with a weak link.

A real bug turned up in the course of measuring this, unrelated to which models were chosen. A library timeout on a streamed request only bounds the gap between chunks, not the whole call — a provider trickling data slowly can run past it without ever tripping it. Caught directly: one call ran 121 seconds and still came back empty, well past the 15-second limit that had been set and never fired. A wall-clock ceiling enforced inside the read loop itself now catches this regardless of which model is asked, or what a provider's connection does.

A typed way in, and a face to go with it

The tray app's first real window. Until now it was a background process with no window at all — a warm dot in the tray, and voice as the only way in when there was no console to type into. A small panel now opens alongside that dot: a typed line in, a transcript out, styled in white and orange after Claude Code's own terminal. Nothing about the conversation itself changed to make this work — the panel satisfies the exact same two interfaces the console already relied on (a thing with `.pending()`/`.take()` for typed input, a thing with `.write()` for what she prints), so `main.py`'s own conversation loop cannot tell a GUI entry box from a keyboard.

Listening is off while this is the way in — not commented out, a config flag (`LISTEN_ENABLED`), the same switch the console app already had. The tray app used to force listening on unconditionally whenever there was no console, on the reasoning that voice was the only way to reach her otherwise; the panel is a second typing surface without a console, so that forcing no longer applies once one is attached. The tray's own **Listening** checkbox still works live, no restart, the moment voice is wanted again.

One real design decision, checked rather than assumed. Tkinter and the tray icon's own event loop both want to own the thread that started them, and getting this wrong would mean a frozen window or a dead tray icon. Verified before writing any of it: the tray library's own documentation states plainly that running its loop off the main thread is unsafe in general — except on Windows specifically, where it's documented as safe. This project is Windows-only throughout, so that's the trade taken: the tray icon gives up the main thread, Tkinter keeps it.

The dot became a cat. Purple, filled when listening, hollow when not — same two states the dot already carried, kept rather than reinvented. The one genuinely new signal is the mouth: open for speaking, closed for idle, replacing a small dark circle inside the old dot that meant the same thing but only by convention. A cat with its mouth open reads as speaking without having to already know what the shape used to mean.

Stage 5 — Losing the graphics card

She moved to a different machine: a Core Ultra 7 255U with twelve cores, 16GB of LPDDR5X-8533, an integrated GPU, an NPU, and no discrete graphics card at all. Everything measured up to this point had been measured on a machine with a 4GB card, and a good deal of the design was shaped around that card's limits — NUM_CTX chosen to fit it, Whisper deliberately kept off it, models sized against it.

The expectation was that she would get slower and that this stage would be about damage. It wasn't, and the reason is worth keeping.

Why a laptop with no graphics card kept up

Generating text is limited by **memory bandwidth**, not by processing power: producing each token means reading the whole model out of memory, so the processor spends most of its time waiting. LPDDR5X-8533 supplies roughly 136 GB/s — the same order as the 4GB card's VRAM. Generation came in at 21 tokens a second, against the roughly 5 a second she can actually speak. Her voice is the bottleneck; the model is not.

What the graphics card was really buying was separation, not speed. It was somewhere to put the language model that wasn't competing with everything else — and that is the thing actually lost here, not throughput.

Giving hearing its own silicon back

The old note beside WHISPER_MODEL said Whisper ran on the CPU so *it wouldn't compete with Ollama for the VRAM*. On the new machine that sentence inverts: with no card, hearing and thinking both wanted the CPU and genuinely fought over it.

But the chip has an NPU — a processor built for fixed-shape neural work — and it was doing nothing at all. Speech recognition is exactly that shape. Whisper moved onto it through OpenVINO, and the separation the design had always assumed was restored on hardware already paid for.

	2.3s	5.4s
	0.55s	0.59s
	0.11s	0.16s

Worth recording that OpenVINO on the CPU measured barely better than what it replaced, and worse on the long clip. The runtime was not the win. Moving off the contended processor was.

A five-second wait that was never the model thinking

Her first reply of a session took 7.68s to begin, and every reply after it took about one. That shape is the tell: nothing about a model gets three times faster on its second use unless something is being cached.

It was. Generation was steady at 21 tok/s the whole time — the delay was her 575-token system prompt being *read*, at about 110 tok/s. Ollama caches the prompt prefix, so the cost is paid once per session and never again.

*The fix was not a shorter prompt. `llm.warm_up()` had been sending an empty message list — which loads the model's weights but never reads a prompt — so the existing prewarm was warming the cheap half and leaving the expensive half for whoever spoke first. It now sends her real system message, during startup, when nobody is waiting. **7.46s to 2.35s.***

The lesson that generalises: both of this stage's real wins came from measuring the two halves of a slow thing separately — reading versus writing, runtime versus processor — rather than treating “it is slow” as one number. Neither fix would have been findable from the total alone, and one of them (a shorter system prompt) would have been the obvious wrong answer.

Not yet built

- Opening or closing applications.
- Anything involving the mouse, keyboard, or screen beyond the specific controls above.
- Browsing the web or clicking anything on your behalf.
- Any action on Linux or macOS — everything in this stage is Windows-specific so far.

Where this goes next is genuinely open. The pattern that's worked a few times now — find the narrowest real API for a specific capability (SMTC for media, Core Audio for volume, a local speaker-embedding model for identity) rather than reaching for something broad and risky — is the one to keep following as this stage grows.